FACULTY OF ARTS

KU LEUVEN

Scripting languages

Lecture 1: Introduction

Tim Van de Cruys



Wednesday 23 September 2026

Introduction

- This course: learning to program, using the Python programming language
- Goal: write your own programs to collect, process and analyze data, with a focus on 'humanities' research: text, tables, data from the web
- No prior programming experience assumed
- Along the way: a way of thinking (computational, algorithmic) that carries over to any other programming language
- But main focus: getting things done

Scripting languages?

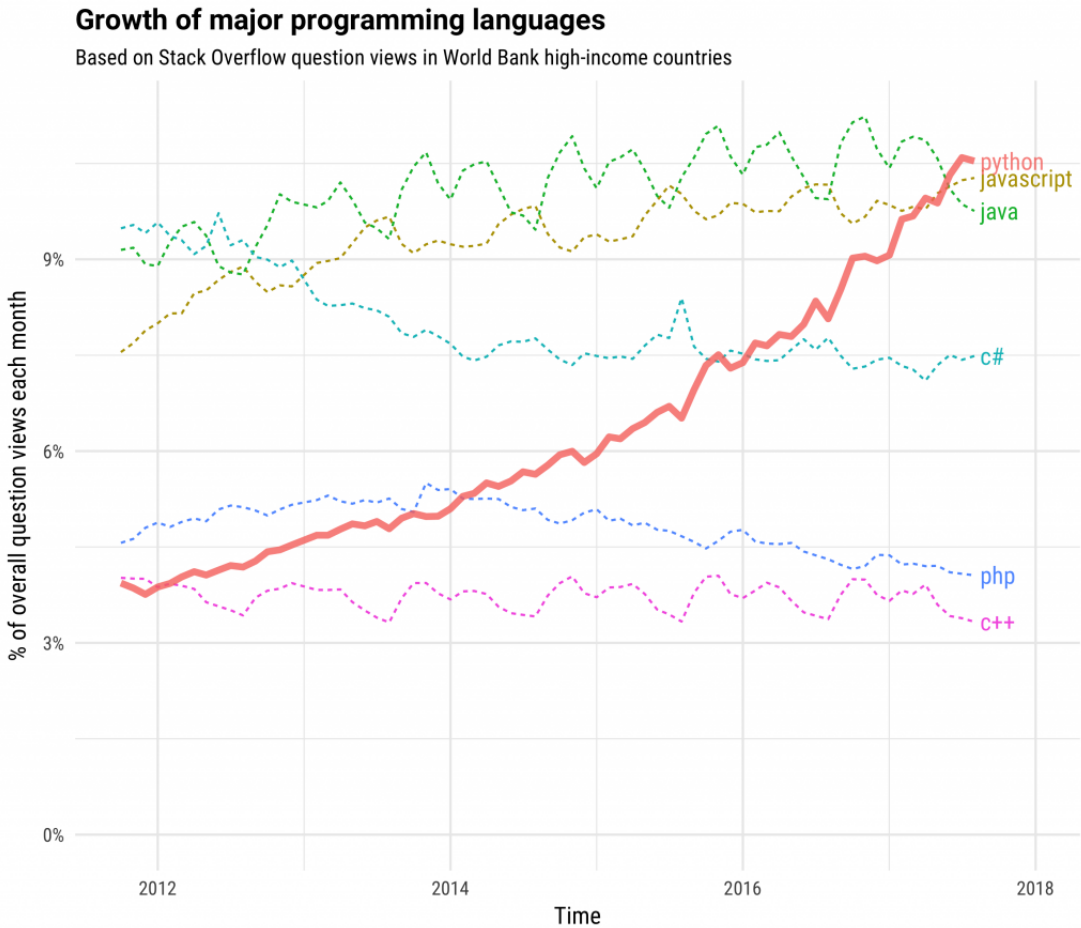
- Programming languages vs. scripting languages
  - Programming language: code needs to be compiled before it can be run
  - Scripting language: code can be executed on the fly, without compilation
  - Difference is becoming less and less important as compilation on modern hardware can often be done very fast, on the fly
- Scripting: light-weight, easy to use piece of code to automate a specific task
  - Hands on attitude to programming
  - Ease of development more important than fast programs
  - Focus on **digital humanities**: scripting for text processing and humanities data

Scripting languages = Python

- In practice, main focus of the course will be: learning to program in Python
- Easy and intuitive language
- Code is easily understandable (similar to plain English)
- Many useful libraries are available (particularly for data science)



Python



Source: `https://stackoverflow.blog/2017/09/06/incredible-growth-python/`

A simple Python program:

```
print('Hello, World!')
```

A simple Python program:

```
print('Hello, World!')
```

The same program in the Java programming language:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Who is in the room?

- Master of Digital Humanities and Master of Sociology (6 ECTS version)
- Master of Artificial Intelligence, Master of Statistics and Data Science, and others (4 ECTS version)
- Different backgrounds
  - No prior programming experience: the lectures start from zero
  - Programmed before: every practical session has more challenging exercises that go beyond a first programming course
- Same lectures, same exam; the 6 ECTS version adds a **project**

Learning goals

- Theoretical
  - Basic competencies in computational and algorithmic thinking
  - Understand the fundamental concepts of programming languages
- **Practical**
  - Develop essential programming skills in Python, with a specific focus on
    - Text processing
    - Exploratory data analysis and visualization
    - Collecting data
  - Work on your own computer as a researcher: files and paths, the command line, environments, running scripts
  - Read, evaluate and debug code, including code written by AI tools

Format of the course

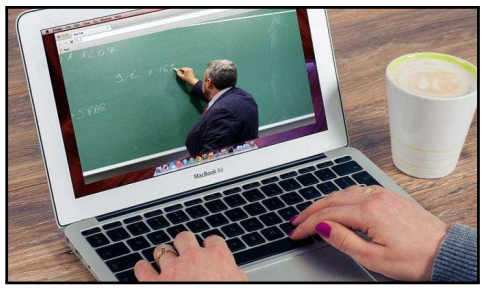
- 12 **lectures** on campus
  - Recording of lectures will be made available on Toledo *if possible*
  - A **code-along notebook** with the examples of each lecture is published on Toledo before class
- Each lecture (except the first one) is accompanied by a **practical session**
  - Self-contained notebook, immediately after the lecture (Wednesday 4pm-6pm)
  - Warm-up exercises, core exercises, and challenge exercises
  - Assistance and questions during the session; solutions published afterwards





Practical information

- Course content (slides + notebooks + reading material + exercises) made available **online on Toledo**
- Possibility to ask questions:
  - Forum of the course on Toledo
  - Q&A during lecture slot and practical session
- Regularly check the Toledo course page!





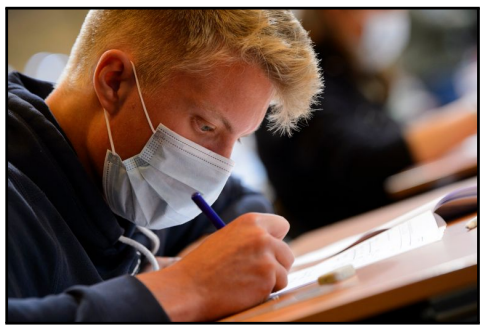


Content of the course

- Introduction; install Python
- Values, variables, strings, lists
- Conditions, loops
- Functions, dictionaries
- Files, paths, the command line, modules
- Errors and exceptions; object-oriented programming
- Text processing
- Regular expressions
- Tabular data analysis and visualization (pandas)
- Collecting data from the web
- DH applications; coding with AI tools

Evaluation

- **4 ECTS**: final exam (20/20)
- **6 ECTS**: final exam ($\frac{2}{3}$) and project ($\frac{1}{3}$)
  - Project: independently apply the techniques of the course to a dataset of your choice
  - Full project brief in lecture 5, so that you can start thinking about a topic early
- Not graded, but strongly recommended: two small **assignments** along the way (with model solutions) and a **mock exam** in the final practical session, so that you know where you stand



The exam

- Computer exam on campus in January (date to be announced)
- Programming exercises, submitted as a notebook
- Exercises test the skills you practise in the practical sessions: writing code, but also reading, explaining and fixing code
- The precise format and the rules on the use of AI tools will be announced during the semester

Computers for DH research?

- Computers allow us
  - to automate (avoid boring work)
  - to amplify (analyze more data than human can handle)
  - to reproduce (share data and code along with publications)

Programming for DH research?

To process data automatically, we can:

- Use existing tools or programs
  - limited to existing functionality
  - often difficult to repeat a sequence of actions
- Create our own programs/scripts
  - Higher investment to get started
  - But endless opportunities

Learning to program in 2026

- Large language models can write Python. So why learn it?
  - Programming knowledge allows you to **specify** your needs more precisely
  - You have to **judge** whether the result is correct: generated code often looks right and is subtly wrong
  - You have to **debug** when it fails, on your data, on your machine
  - You cannot take **responsibility** when you have no idea what goes on under the hood!
- Advice for the first weeks: type the code yourself, and use AI tools to *explain*, not to *solve*

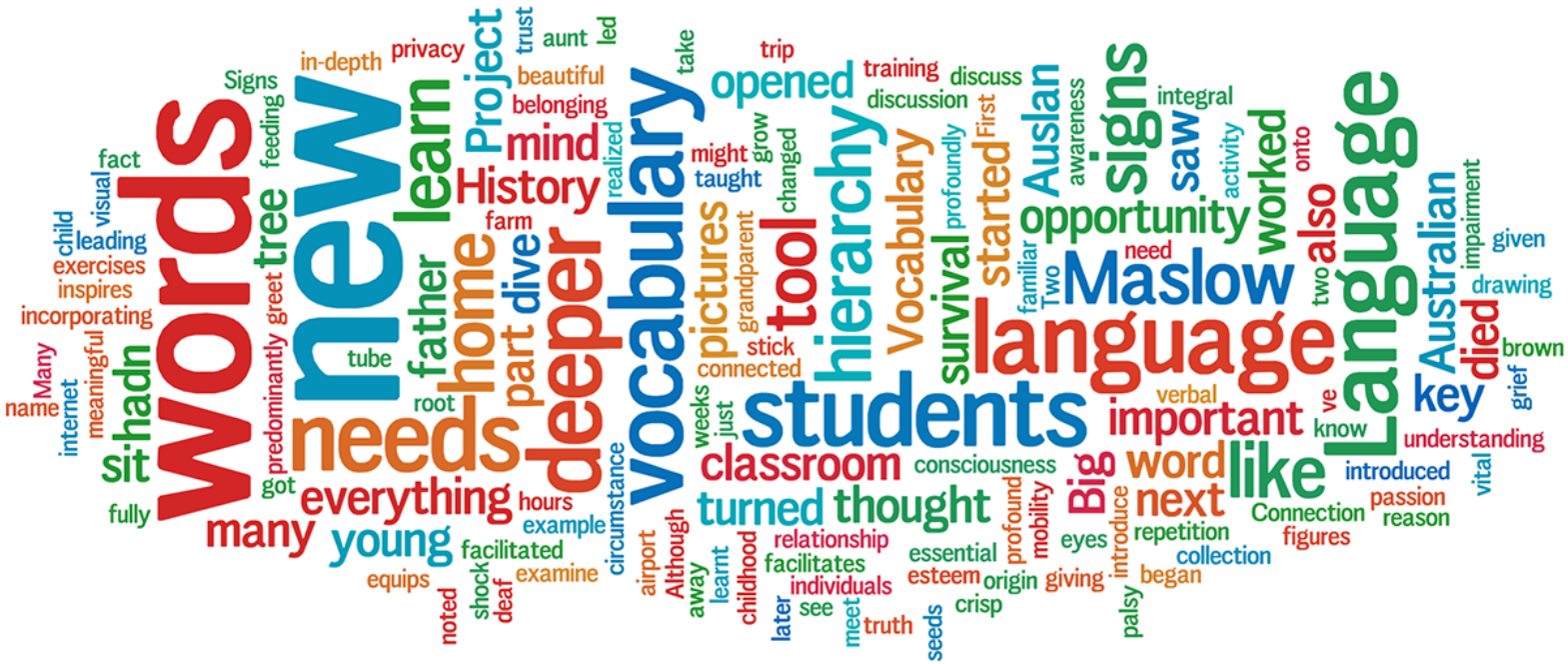
Python is a language

A language has:

- **words**: primitive constructs, basic building blocks of the language
- **syntax**: rules that stipulate how words combine
- **semantics**: the meaning that is derived from the words and the way they are combined

Words

English



Words

Python

```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

Words

Python

```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

- Also numbers and simple operators (+, -, *)

Words

Python

```
help> keywords

Here is a list of the Python keywords.  Enter any keyword to get more help.

False               class               from                or
None                continue            global              pass
True                def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
break               for                 not
```

- Also numbers and simple operators (+, -, *)
- And Python allows you to invent your own words, which have meaning to you (called *variables*)

Syntax

English

Some expressions are syntactically valid, while others are not:

Syntax

English

Some expressions are syntactically valid, while others are not:

- the dog chases the cat
- cat library the sleep bottle

Syntax

English

Some expressions are syntactically valid, while others are not:

- the dog chases the cat $\rightarrow$ syntactically valid
- cat library the sleep bottle $\rightarrow$ NOT syntactically valid

Syntax

Python

Some expressions are syntactically valid, while other are not:

- `2.5 * 14`
- `"welcome" 2.5 in`

Syntax

Python

Some expressions are syntactically valid, while other are not:

- `2.5 * 14` → syntactically valid
- `"welcome" 2.5 in` → NOT syntactically valid

Semantics

English

Certain expressions are syntactically valid, but are non-sensical:

Semantics

English

Certain expressions are syntactically valid, but are non-sensical:

- the man reads the newspaper
- the lightbulb sings a banana

Semantics

English

Certain expressions are syntactically valid, but are non-sensical:

- the man reads the newspaper $\rightarrow$ semantically felicitous
- the lightbulb sings a banana $\rightarrow$ semantically anomalous

Semantics

Python

Certain expressions are syntactically valid, but are non-sensical:

- $2 + 5$
- `"welcome" + 1`

Semantics

Python

Certain expressions are syntactically valid, but are non-sensical:

- $2 + 5 \rightarrow$ semantically felicitous
- `"welcome" + 1` $\rightarrow$ semantically anomalous

Semantics (bis)

English

Certain expressions can be ambiguous:

- I saw the man with the telescope

Semantics (bis)

Python

- Expressions only have one single meaning
- But the meaning may be different from what the programmer intended

Semantics (bis)

Python

- Expressions only have one single meaning
- But the meaning may be different from what the programmer intended
- `2 + 5 * 10`

Semantics (bis)

Python

- Expressions only have one single meaning
- But the meaning may be different from what the programmer intended
- `(2 + 5) * 10`

Semantics (bis)

Python

- Expressions only have one single meaning
- But the meaning may be different from what the programmer intended
- `2 + (5 * 10)`

Learning a language

- Learning Python is like learning a language
- You will (and need to!) make mistakes
  - Syntactic errors
    - common and easily caught
  - Semantic errors
    - Python will check for significant semantic anomalies
    - Subtle semantic differences can cause unexpected behaviour
  - Ambiguity
    - No error, but meaning is different from what programmer intended
    - Script crashes, script runs forever, script returns a different answer than expected, ...

Python script

- a **script** is a sequence of definitions and commands
  - definitions are evaluated
  - commands are executed by Python interpreter in a shell
- **commands** (statements) instruct interpreter to do something
- can be typed directly in a shell or stored in a file that is read into the shell and evaluated

Objects

- A script manipulates **data objects**
- Objects have a **type** that defines the kinds of things programs can do to them
  - a banana is of type *fruit* so it can be eaten, peeled, etc.
  - Kim is of type *human* so she can walk, speak, etc.
- Objects are:
  - scalar (cannot be subdivided)
  - non-scalar (have internal structure that can be accessed)

How to run Python code?

- Two-step process: edit code, then run
  - Write entire script, and save in a file
  - Give file to Python interpreter to execute
  - (Solve errors and repeat)
- Interactive shell
  - Launch interactive Python shell
  - Type commands directly into shell
- Notebook
  - Most interactive way to run Python code
  - Notebook combines explanatory text, code, and results
  - Interface through web browser
  - User-friendly version provided by Google: Google Colab (`https://colab.research.google.com`)

Installing Python

- Main Python release
  - Available at `http://www.python.org`
  - Can be somewhat cumbersome to use, particularly under Windows
- Anaconda distribution (recommended)
  - Available at `https://www.anaconda.com/download`
  - User-friendly distribution
  - Suitable for data science and machine learning
  - Simplifies package management and code execution
  - Comes with Jupyter Notebook, which we use in the practical sessions
- Note: installation process (and which installer to download) depends on your operating system

Reference material

- Python for Everybody: `https://www.py4e.com/`
- Charles Severance
- Book, with focus on Python for data processing
- Website comes with additional videos, exercises, ...
- Creative commons license

For next week

- Install Python on your own computer
  - Practical sessions: recommended to use your own laptop
  - If not possible: use Google Colab
- Read chapters 1 & 2 of Python for Everybody